

FREEDOM INTERNATIONAL SCHOOL

COMPUTER SCIENCE (083)

2026-27

STRING BUILT IN FUNCTIONS

1. The **capitalize()** method returns a string where the first character is upper case, and the rest is lower case.
2. The **title()** method returns a string where the first character in every word is converted to upper case and other characters in the word are converted to lowercase. Like a header, or a title. If the word contains a number or a symbol, the first letter after that will be converted to upper case. But in capitalize if the first letter in a sentence is a number then capitalize string is as the original string.
3. The **count()** method returns the number of times a specified value appears in the string. *string.count(value, start, end)*. Start and end parameters are optional.
4. The **endswith()** method returns True if the string ends with the specified value, otherwise False. *string.endswith(value, start, end)*. Start and end parameters are optional.
5. The **startswith()** method returns True if the string starts with the specified value, otherwise False.
6. The **find()** method finds the first occurrence of the specified value. The find() method returns -1 if the value is not found. *string.find(value, start, end)*. Start and end parameters are optional
7. The **index()** method finds the first occurrence of the specified value. The index() method raises an exception if the value is not found. The index() method is almost the same as the find() method, the only difference is that the find() method returns -1 if the value is not found and the index() method raises an exception if the value is not found. *string.index(value, start, end)*. Start and end parameters are optional
8. The **isalnum()** method returns True if all the characters are alphanumeric, meaning alphabet letter (a-z) and numbers (0-9). Example of characters that are not alphanumeric: (space)!#%&? etc. *string.isalnum()*
9. The **isalpha()** method returns True if all the characters are alphabet letters (a-z). *string.isalpha()*
10. The **isdigit()** method returns True if all the characters are digits, otherwise False. *string.isdigit()*

11. The **isnumeric()** method returns True if all the characters are numeric (0-9), otherwise False. Exponents, like ² and ³/₄ are also considered to be numeric values. "-1" and "1.5" are NOT considered numeric values, because *all* the characters in the string must be numeric, and the - and the . are not. *string.isnumeric()*
12. The **islower()** method returns True if all the characters are in lower case, otherwise False. *string.islower()*
13. The **isupper()** method returns True if all the characters are in upper case, otherwise False. Numbers, symbols and spaces are not checked, only alphabet characters. *string.isupper()*
14. The **isspace()** method returns True if all the characters in a string are whitespaces, otherwise False.
15. The **istitle()** method returns True if all words in a text start with a upper case letter, AND the rest of the word are lower case letters, otherwise False. Symbols and numbers are ignored. *string.istitle()*
16. The **join()** method takes all items in an iterable and joins them into one string. A string must be specified as the separator. *string.join(iterable)*
17. The **lower()** method returns a string where all characters are lower case. Symbols and Numbers are ignored. *string.lower()*
18. The **lstrip()** method removes any leading characters (space is the default leading character to remove). *string.lstrip(characters)*. Characters are optional
19. The **rstrip()** method removes any trailing characters (characters at the end a string), space is the default trailing character to remove.
20. The **partition()** method searches for a specified string, and splits the string into a tuple containing three elements. The first element contains the part before the specified string. The second element contains the specified string. The third element contains the part after the string. **Note:** This method searches for the *first* occurrence of the specified string. *string.partition(value)*
21. The **split()** method splits a string into a list. You can specify the separator, default separator is any whitespace. *string.split(separator, maxsplit)*. Separator is Optional. Specifies the separator to use when splitting the string. By default any whitespace is a separator. Maxsplit is Optional. Specifies how many splits to do. Default value is -1, which is "all occurrences". Note: When maxsplit is specified, the list will always contain the specified number of elements plus one.
22. The **replace()** method replaces a specified phrase with another specified phrase. **Note:** All occurrences of the specified phrase will be replaced, if nothing else is specified.

string.replace(oldvalue, newvalue, count). Count is Optional. A number specifying how many occurrences of the old value you want to replace. Default is all occurrences

LIST BUILT IN FUNCTIONS

1. **Append-** is used to add a single item to the end of list. This method modifies the original list and does not return a new list. When appending one list to another, the entire list is added as a single element, creating a nested list. Syntax- `a.append(element)`
2. **Extend-** is used to add items from one list to the end of another list. This method modifies the original list by appending all items from the given iterable. Syntax- `a.extend(iterable)`. It doesn't return anything just modifies the original list.
3. **Clear-** is used to remove all elements from a list, effectively making it an empty list. This method does not delete the list itself but clears its content. Syntax- `a.clear()`
4. **Count-** is used to find the number of times a specific element occurs in a list. Syntax- `a.count(element)`
5. **Index-** is a helpful tool when you want to find the position of a specific item in a list. It works by searching through the list from the beginning and returning the index (position) of the first occurrence of the element you're looking for. Syntax- `a.index(element, start, end)`. It raises `ValueError` if the element is not found.
6. **Insert-** inserts an element at a specific index in the list. Syntax- `a.insert(index, element)` . If the index mentioned doesn't exist in the list then element is inserted in the end. Again doesn't return anything but modifies original list.
7. **Pop-** removes and returns an element from a list. By default, it removes the last item, but we can specify an index to remove a particular element. It directly modifies the original list. Syntax-`a.pop(index)` index is optional. If index mentioned doesn't exist `IndexError` is raised.
8. **Remove-** removes the first occurrence of a given item from list. It make changes to the current list. It only takes one argument, element we want to remove and if that element is not present in the list, it gives **ValueError**. The method does not return any value but removes the given object from the list. Syntax- `a.remove(element)`
9. **Reverse-** that reverses the order of elements in a list. This method modifies the original list and does not return a new list. Syntax- `a.reverse()`. The **reverse()** method is specific to **lists** in Python (it's an attribute of a list). Therefore, attempting to use

the **reverse()** method on other data structures, such as strings, tuples, sets, or dictionaries, will result in an error.

10. **Sort-** is used to arrange the elements of a list in a specific order, either ascending or descending. It changes the original list directly, so no new list is created. Syntax - `a.sort(reverse=True)` if reverse is True it sorts in descending, reverse not used will sort in ascending.
11. **Sorted-** is used to arrange elements of any iterables in specific order and returns iterable sorted in a particular order in the form of list and the original iterable is intact.